

第 18 章

视频处理

视频信号（以下简称为视频）是非常重要的视觉信息来源，它是视觉处理过程中经常要处理的一类信号。实际上，视频是由一系列图像构成的，这一系列图像被称为帧，帧是以固定的时间间隔从视频中获取的。获取（播放）帧的速度称为帧速率，其单位通常使用“帧/秒”表示，代表在 1 秒内所出现的帧数，对应的英文是 FPS（Frames Per Second）。如果从视频中提取出独立的帧，就可以使用图像处理的方法对其进行处理，达到处理视频的目的。

OpenCV 提供了 `cv2.VideoCapture` 类和 `cv2.VideoWriter` 类来支持各种类型的视频文件。在不同的操作系统中，它们支持的文件类型可能有所不同，但是在各种操作系统中均支持 AVI 格式的视频文件。

本章主要介绍 `cv2.VideoCapture` 类和 `cv2.VideoWriter` 类的相关函数，并讲解应用它们进行捕获摄像头文件、播放视频文件、保存视频等基础操作。

18.1 VideoCapture 类

OpenCV 提供了 `cv2.VideoCapture` 类来处理视频。`cv2.VideoCapture` 类处理视频的方式非常简单、快捷，而且它既能处理视频文件又能处理摄像头信息。

18.1.1 类函数介绍

`cv2.VideoCapture` 类的常用函数包括初始化、打开、帧捕获、释放、属性设置等，下面对这些函数进行简单的介绍。

1. 初始化

OpenCV 为 `cv2.VideoCapture` 类提供了构造函数 `cv2.VideoCapture()`，用于打开摄像头并完成摄像头的初始化工作。该函数的语法格式为：

捕获对象=`cv2.VideoCapture("摄像头 ID 号")`

式中：

- “摄像头 ID 号”就是摄像头的 ID 号码。需要注意的是，这个参数是摄像设备（摄像头）的 ID 编号，而不是文件名。其默认值为 -1，表示随机选取一个摄像头；如果有多个摄像头，则用数字“0”表示第 1 个摄像头，用数字“1”表示第 2 个摄像头，以此类推。所以，如果只有一个摄像头，既可以使用“0”，也可以使用“-1”作为摄像头 ID 号。



在某些平台上，如果该参数值为“-1”，OpenCV 会弹出一个窗口，让用户手动选择希望使用的摄像头。

- “捕获对象”为返回值，是 `cv2.VideoCapture` 类的对象。

例如，要初始化当前的摄像头，可以使用语句：

```
cap = cv2.VideoCapture(0)
```

OpenCV 官网在介绍函数 `cv2.VideoCapture()` 时，特别强调：视频处理完以后，要记得释放摄像头对象。

该（构造）函数也能够用于初始化视频文件，初始化视频文件时，参数为文件名。此时函数的形式为：

```
捕获对象=cv2.VideoCapture("文件名")
```

例如，打开当前目录下文件名为“vtest.avi”的视频文件，可以使用语句：

```
cap = cv2.VideoCapture('vtest.avi')
```

2. `cv2.VideoCapture.open()` 函数和 `cv2.VideoCapture.isOpened()` 函数

一般情况下，使用 `cv2.VideoCapture()` 函数即可完成摄像头的初始化。有时，为了防止初始化发生错误，可以使用函数 `cv2.VideoCapture.isOpened()` 来检查初始化是否成功。该函数的语法格式为：

```
retval = cv2.VideoCapture.isOpened()
```

该函数会判断当前的摄像头是否初始化成功：

- 如果成功，则返回值 `retval` 为 `True`。
- 如果不成功，则返回值 `retval` 为 `False`。

如果摄像头初始化失败，可以使用函数 `cv2.VideoCapture.open()` 打开摄像头。该函数的语法格式为：

```
retval = cv2.VideoCapture.open( index )
```

式中：

- `index` 为摄像头 ID 号。
- `retval` 为返回值，当摄像头（或者视频文件）被成功打开时，返回值为 `True`。

同样，函数 `cv2.VideoCapture.isOpened()` 和函数 `cv2.VideoCapture.open()` 也能用于处理视频文件。在处理视频文件时，函数 `cv2.VideoCapture.open()` 的参数为文件名，其语法格式为：

```
retval = cv2.VideoCapture.open( filename )
```

3. 捕获帧

摄像头初始化成功后，就可以从摄像头中捕获帧信息了。捕获帧所使用的是函数 `cv2.VideoCapture.read()`。该函数的语法是：

```
retval, image=cv2.VideoCapture.read()
```

式中：

- image 是返回的捕获到的帧，如果没有帧被捕获，则该值为空。
- retval 表示捕获是否成功，如果成功则该值为 True，不成功则为 False。

4. 释放

在不需要摄像头时，要关闭摄像头。关闭摄像头使用的是函数 cv2.VideoCapture.release()。该函数的语法是：

```
None=cv2.VideoCapture.release()  
例如，当前有一个 VideoCapture 类的对象 cap，要将其释放，可以使用语句：  
cap.release()
```

5 属性设置

有时，我们需要获取 cv2.VideoCapture 类对象的属性，或是更改该类对象的属性。函数 cv2.VideoCapture.get()用于获取 cv2.VideoCapture 类对象的属性，该函数的语法格式是：

```
retval = cv2.VideoCapture.get( propId )
```

式中，参数 propId 对应着 cv2.VideoCapture 类对象的属性。

例如，有一个 cv2.VideoCapture 类对象 cvc，则：

- 通过 cvc.get(cv2.CAP_PROP_FRAME_WIDTH)，就能获取当前帧对象的宽度。
- 通过 cvc.get(cv2.CAP_PROP_FRAME_HEIGHT)，就能获取当前帧对象的高度。

函数 cv2.VideoCapture.set()用来设置 cv2.VideoCapture 类对象的属性。该函数的语法是：

```
retval = cv2.VideoCapture.set( propId, value )
```

式中，propId 对应 cv2.VideoCapture 类对象的属性，value 对应属性 propid 的值。

例如，有一个 cv2.VideoCapture 类对象 cvc，则：

- 语句 ret = cvc.set(cv2.CAP_PROP_FRAME_WIDTH, 640)将当前帧对象的宽度设置为 640 像素。
- 语句 ret = cvc.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)将当前帧对象的高度设置为 480 像素。

cv2.VideoCapture 类对象的属性值及含义如表 18-1 所示。

表 18-1 属性值及含义¹

值	propId	含义
cv2.CAP_PROP_POS_MSEC	0	视频文件的当前位置（以 ms 为单位）或视频被捕获时的时间戳
cv2.CAP_PROP_POS_FRAMES	1	接下来要解码/捕获的帧的索引从 0 开始
cv2.CAP_PROP_POS_AVI_RATIO	2	视频文件的相对位置：0 表示视频的开始，1 表示视频的结束
cv2.CAP_PROP_FRAME_WIDTH	3	帧的宽度
cv2.CAP_PROP_FRAME_HEIGHT	4	帧的高度
cv2.CAP_PROP_FPS	5	帧速

1 表格中空白列均表示官方未做出明确说明。

续表

值	propId	含义
cv2.CAP_PROP_FOURCC	6	用 4 个字符表示的视频编码器格式
cv2.CAP_PROP_FRAME_COUNT	7	帧数
cv2.CAP_PROP_FORMAT	8	byretrieve()返回的 Mat 格式的对象
cv2.CAP_PROP_MODE	9	用于表明当前捕获模式的后端特有的值
cv2.CAP_PROP_BRIGHTNESS	10	图像的亮度（仅适用于相机）
cv2.CAP_PROP_CONTRAST	11	图像对比度（仅适用于相机）
cv2.CAP_PROP_SATURATION	12	图像饱和度（仅适用于相机）
cv2.CAP_PROP_HUE	13	图像色相（仅适用于相机）
cv2.CAP_PROP_GAIN	14	图像增益（仅适用于相机）
cv2.CAP_PROP_EXPOSURE	15	曝光（仅适用于相机）
cv2.CAP_PROP_CONVERT_RGB	16	表示图像是否应转换为 RGB 的逻辑标志
cv2.CAP_PROP_WHITE_BALANCE_BLUE_U	17	当前不支持
cv2.CAP_PROP_RECTIFICATION	18	立体摄像机的整流标志
cv2.CAP_PROP_MONOCHROME	19	
cv2.CAP_PROP_SHARPNESS	20	
cv2.CAP_PROP_AUTO_EXPOSURE	21	DC1394: 相机的曝光控制，用户仅仅能通过该属性调整
cv2.CAP_PROP_GAMMA	22	
cv2.CAP_PROP_TEMPERATURE	23	
cv2.CAP_PROP_TRIGGER	24	
cv2.CAP_PROP_TRIGGER_DELAY	25	
cv2.CAP_PROP_WHITE_BALANCE_RED_V	26	
cv2.CAP_PROP_ZOOM	27	
cv2.CAP_PROP_FOCUS	28	
cv2.CAP_PROP_GUID	29	
cv2.CAP_PROP_ISO_SPEED	30	
cv2.CAP_PROP_BACKLIGHT	31	
cv2.CAP_PROP_PAN	32	
cv2.CAP_PROP_TILT	33	
cv2.CAP_PROP_ROLL	34	
cv2.CAP_PROP_IRIS	35	
cv2.CAP_PROP_SETTINGS	36	弹出过滤对话框
cv2.CAP_PROP_BUFFERSIZE	37	
cv2.CAP_PROP_AUTOFOCUS	38	
cv2.CAP_PROP_SAR_NUM	39	num/den (num)
cv2.CAP_PROP_SAR_DEN	40	num/den (den)

5. cv2.VideoCapture.grab()函数和cv2.VideoCapture.retrieve()函数

一般情况下，如果需要读取一个摄像头的视频数据，最简便的方法就是使用函数 `cv2.VideoCapture.read()`。但是，如果需要同步一组或一个多头（multihead）摄像头（例如立体摄像头或 Kinect）的视频数据时，该函数就无法胜任了。可以把函数 `cv2.VideoCapture.read()` 理解为是由函数 `cv2.VideoCapture.grab()` 和函数 `cv2.VideoCapture.retrieve()` 组成的。函数 `cv2.VideoCapture.grab()` 用来指向下一帧，函数 `cv2.VideoCapture.retrieve()` 用来解码并返回一帧。



因此, 可以使用函数 `cv2.VideoCapture.grab()` 和函数 `cv2.VideoCapture.retrieve()` 获取多个摄像头的的数据。

函数 `cv2.VideoCapture.grab()` 用来指向下一帧, 其语法格式是:

```
retval= cv2.VideoCapture.grab( )
```

如果该函数成功指向下一帧, 则返回值 `retval` 为 `True`。

函数 `cv2.VideoCapture.retrieve()` 用来解码, 并返回函数 `cv2.VideoCapture.grab()` 捕获的视频帧。该函数的语法格式为:

```
retval, image = cv2.VideoCapture.retrieve( )
```

式中:

- `image` 为返回的视频帧, 如果未成功, 则返回一个空图像。
- `retval` 为布尔型值, 若未成功, 返回 `False`; 否则, 返回 `True`。

对于一组摄像头, 可以使用如下代码捕获不同摄像头的视频帧:

```
success0 = cameraCapture0.grab()
success1 = cameraCapture1.grab()
if success0 and success1:
    frame0 = cameraCapture0.retrieve()
    frame1 = cameraCapture1.retrieve()
```

与 `VideoCapture` 类内的其他函数一样, `cv2.VideoCapture.grab()` 和 `cv2.VideoCapture.retrieve()` 也能用来读取视频文件。

18.1.2 捕获摄像头视频

计算机视觉要处理的对象是多种多样的。有时, 我们需要处理的可能是某个特定的图像; 有时, 要处理的可能是磁盘上的视频文件; 而在更多时候, 要处理的是从摄像设备中实时读入的视频流。

OpenCV 通过 `cv2.VideoCapture` 类提供了非常方便的捕获摄像头视频的方法。下面用一个例子介绍如何捕获摄像头视频。

【例 18.1】 使用 `cv2.VideoCapture` 类捕获摄像头视频。

根据题目要求, 编写代码如下:

```
import numpy as np
import cv2
cap = cv2.VideoCapture(0)
while(cap.isOpened()):
    ret, frame = cap.read()
    cv2.imshow('frame', frame)
    c = cv2.waitKey(1)
    if c==27: #ESC 键
        break
cap.release()
```



```
cv2.destroyAllWindows()
```

运行程序，笔者拿着一本 OpenCV 图书放在摄像头前，程序捕获的摄像头视频的截图如图 18-1 所示。



图 18-1 捕获的摄像头视频的截图

18.1.3 播放视频文件

播放视频文件时，需要将函数 `cv2.VideoCapture()` 的参数值设置为视频文件的名称。在播放视频时，可以通过设置函数 `cv2.waitKey()` 中的参数值，来设置播放视频时每一帧的持续（停留）时间。如果函数 `cv2.waitKey()` 中的参数值：

- 较小，则说明每一帧停留的时间较短，视频播放速度会较快。
- 较大，则说明每一帧停留的时间较长，视频播放速度会较慢。

该参数的单位是 ms，通常情况下，将这个参数的值设置为 25 即可。

【例 18.2】 使用 `cv2.VideoCapture` 类播放视频文件。

根据题目的要求，编写代码如下：

```
import numpy as np
import cv2
cap = cv2.VideoCapture('viptrain.avi')
while(cap.isOpened()):
    ret, frame = cap.read()
    cv2.imshow('frame', frame)
    c = cv2.waitKey(25)
    if c==27: #ESC 键
        break
cap.release()
cv2.destroyAllWindows()
```

运行上述程序，播放的视频的截图如图 18-2 所示。

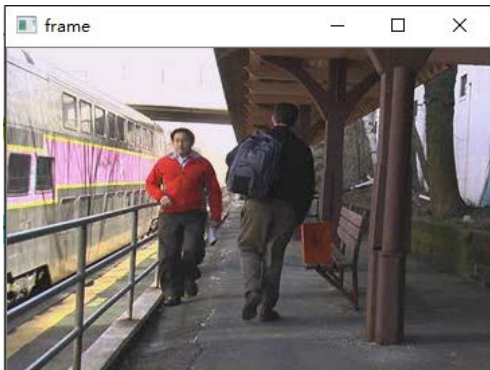


图 18-2 【例 18.2】程序播放的视频的截图

18.2 VideoWriter 类

OpenCV 中的 `cv2.VideoWriter` 类可以将图片序列保存成视频文件，也可以修改视频的各种属性，还可以完成对视频类型的转换。

18.2.1 类函数介绍

`cv2.VideoWriter` 类常用的成员函数包括：构造函数、`write` 函数等。本节简单介绍这两个常用的函数。

1. 构造函数

OpenCV 为 `cv2.VideoWriter` 类提供了构造函数，用它来实现初始化工作。该函数的语法格式是：

```
<VideoWriter object> = cv2.VideoWriter( filename, fourcc, fps, frameSize[, isColor] )
```

式中：

- `filename` 指定输出目标视频的存放路径和文件名。如果指定的文件名已经存在，则会覆盖这个文件。
- `fourcc` 表示视频编/解码类型（格式）。在 OpenCV 中用函数 `cv2.VideoWriter_fourcc()` 来指定视频编码格式。`cv2.VideoWriter_fourcc()` 有 4 个字符参数。这 4 个字符参数构成了编/解码器的“4 字标记”，每个编/解码器都有一个这样的标记。下面列出几个常用的标记。
 - `cv2.VideoWriter_fourcc('I', '4', '2', '0')` 表示未压缩的 YUV 颜色编码格式，色度子采样为 4:2:0。该编码格式具有较好的兼容性，但产生的文件较大，文件扩展名为 `.avi`。
 - `cv2.VideoWriter_fourcc('P', 'T', 'M', 'T')` 表示 MPEG-1 编码类型，生成的文件的扩展名为 `.avi`。
 - `cv2.VideoWriter_fourcc('X', 'V', 'I', 'D')` 表示 MPEG-4 编码类型。如果希望得到的视频大小为平均值，可以选用这个参数组合。该组合生成的文件的扩展名为 `.avi`。

- `cv2.VideoWriter_fourcc('T', 'H', 'E', 'O')`表示 Ogg Vorbis 编码类型，文件的扩展名为.ogv。
- `cv2.VideoWriter_fourcc('F', 'L', 'V', 'T')`表示 Flash 视频，生成的文件的扩展名为.flv。

欲知更多的字符参数组合，可以在网站 <http://www.fourcc.org> 上查询。若参数 `fourcc` 为“-1”，则程序运行时会弹出一个对话框，如图 18-3 所示。在该对话框中，用户可以根据自己的需要选择合适的压缩程序和压缩质量。

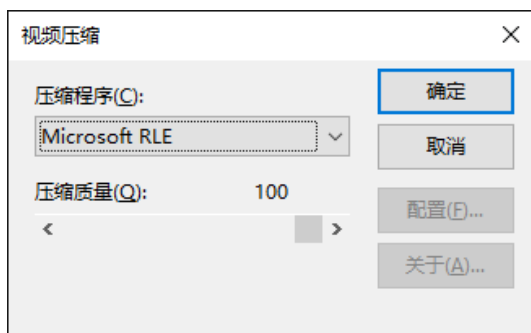


图 18-3 “视频压缩”对话框

- `fps` 为帧速率。
- `frameSize` 为每一帧的长和宽。
- `isColor` 表示是否为彩色图像。

例如，下面的语句完成了 `cv2.VideoWriter` 类的初始化工作：

```
fourcc = cv2.VideoWriter_fourcc(*'XVID')
out = cv2.VideoWriter('output.avi', fourcc, 20, (1024, 768))
```

如果希望通过如图 18-3 所示对话框设置编/解码格式，可以使用语句：

```
fourcc = -1
out = cv2.VideoWriter('output.avi', fourcc, 20, (1024, 768))
```

2. write函数

`cv2.VideoWriter` 类中的函数 `cv2.VideoWriter.write()`用于写入下一帧视频。该函数的语法格式为：

```
None=cv2.VideoWriter.write(image)
```

式中，`image` 是要写入的视频帧。通常情况下，要求彩色图像的格式为 BGR 模式。

在调用该函数时，直接将要写入的视频帧传入该函数即可。例如，有一个视频帧为 `frame`，要将其写入上面的示例中名为 `out` 的 `cv2.VideoWriter` 类对象内，则使用语句：

```
out.write(frame)
```

上述语句会把 `frame` 传入名为 `output.avi` 的 `out` 对象内。

3. 释放

在不需要 `cv2.VideoWriter` 类对象时，需要将其释放。释放该类对象时使用的是函数 `cv2.VideoWriter.release()`。该函数的语法格式是：


```
None = cv2.VideoWriter.release( )
```

例如，当前有一个 cv2.VideoWriter 类的对象 out，可以用以下语句将其释放：

```
out.release()
```

18.2.2 保存视频

保存视频包括创建对象、写入视频、释放对象等多个步骤，下面对各个步骤做简单的介绍。

1. 创建对象

在创建对象前，首先需要设置好参数。

- 设置好要保存的具体文件名，例如：filename="out.avi"。
- 使用 cv2.VideoWriter_fourcc() 确定编 / 解码的类型，例如：fourcc = cv2.VideoWriter_fourcc(*'XVID')。
- 确定视频的帧速率，例如：fps=20。
- 确定视频的长度和宽度，例如：size=(640,480)。

然后利用上述参数，创建对象。例如：

```
out = cv2.VideoWriter( filename , fourcc , fps , size )
```

当然，也可以直接在构造函数内用需要的参数值创建对象。例如：

```
out = cv2.VideoWriter('out.avi',fourcc, 20, (640,480))
```

2. 写入视频

用函数 cv2.VideoWriter.write() 在创建的对象 out 内写入读取到的视频帧 frame。使用的代码为：

```
out.write(frame)
```

3. 释放对象

在完成写入后，释放对象 out。代码为：

```
out.release()
```

【例 18.3】 使用 cv2.VideoWriter 类保存摄像头视频文件。

根据题目的要求，编写代码如下：

```
import numpy as np
import cv2
cap = cv2.VideoCapture(0)
fourcc = cv2.VideoWriter_fourcc('I','4','2','0')
out = cv2.VideoWriter('output.avi',fourcc, 20, (640,480))
while(cap.isOpened()):
    ret, frame = cap.read()
    if ret==True:
        out.write(frame)
        cv2.imshow('frame',frame)
```



```

        if cv2.waitKey(1) == 27:
            break
    else:
        break
cap.release()
out.release()
cv2.destroyAllWindows()

```

运行上述程序，程序就会捕获当前摄像头的视频内容，并将其保存在当前目录下名为“output.avi”的视频文件中。

18.3 视频操作基础

视频是由视频帧构成的，将视频帧从视频中提取出，对其使用图像处理的方法进行处理，就可以达到处理视频的目的。

【例 18.4】提取视频的 Canny 边缘检测结果。

根据题目的要求，设计代码如下：

```

import numpy as np
import cv2
cap = cv2.VideoCapture('viptrain.avi')
while(cap.isOpened()):
    ret, frame = cap.read()
    frame=cv2.Canny(frame,100,200)
    cv2.imshow('frame',frame)
    c = cv2.waitKey(1)
    if c==27: #ESC 键
        break
cap.release()
cv2.destroyAllWindows()

```

运行程序，就会播放对应视频的 Canny 边缘检测结果，视频播放的截图如图 18-4 所示。



图 18-4 播放视频的边缘检测结果